

LAPORAN PRAKTIKUM 2
PEMOGRAMAN BERORIENTASI OBJEK

Disusun Oleh:

Dinda Amelia

(2511531020)

Dosen Pengampu:

Nurfiah, S.ST, M.Kom.

Asisten Praktikum:

Ezza Addini



DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
PADANG 2026

KATA PENGANTAR

Segala puji bagi Allah SWT yang telah memberikan kesempatan serta kemudahan kepada penulis untuk dapat menyelesaikan Laporan Praktikum pada mata kuliah Struktur Data, sehingga Laporan Praktikum ini dapat dikumpulkan dengan tepat waktu. Atas rahmat dan karunianya Laporan Praktikum dapat terselesaikan dengan baik. Sholawat serta salam juga penulis sampaikan kepada baginda tercinta yaitu Nabi Muhammad SAW, yang kita nantikan syafa'atnya di akhirat nanti. Laporan Praktikum ini bertujuan untuk menambah wawasan para pembaca untuk lebih memperdalam ilmu yang ada pada makalah ini.

Laporan pratikum ini disusun sebagai bentuk penanggung jawaban atas pelaksanaan kegiatan pratikum mata kuliah Pemograman Berorientasi Objek yang membahas tentang konsep Relasi Antar Objek (Class Diagram & Asosiasi).

Dalam penyusunan Laporan Praktikum ini, penulis mengalami banyak kesulitan dan penulis menyadari bahwa Laporan Praktikum ini masih jauh dari kesempurnaan. Untuk itu, penulis sangat mengharapkan kritik dan saran yang membangun demi kesempurnaan pada Laporan Praktikum ini.

Padang, 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI	ii
BAB I	1
PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan.....	1
1.3 Manfaat.....	1
1.4 Link repository Github:	2
BAB II.....	3
PEMBAHASAN	3
2.1 Dasar Teori	3
2.2 Langkah kerja	4
2.2.1 Class 1 “Transaksi”	4
2.2.2 Class 2 “Rekening”	6
2.2.3 Class 3 “Main”.....	13
BAB III	21
KESIMPULAN	21
DAFTAR PUSTAKA	22

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam pengembangan perangkat lunak berbasis objek (*Object-Oriented Programming* / OOP), dunia nyata dimodelkan ke dalam bentuk objek-objek yang saling berinteraksi. Pada modul sebelumnya, sistem keuangan bank yang dibuat hanya mampu mengubah saldo saat proses setor dan tarik tanpa menyimpan riwayat aktivitas tersebut. Padahal, dalam aplikasi perbankan sebenarnya, pencatatan jejak transaksi sangat krusial untuk transparansi dan pembuatan laporan mutasi rekening.

Oleh karena itu, diperlukan penerapan Relasi Asosiasi (*one-to-many*) dan Struktur Data Dinamis (*ArrayList*). Dengan pendekatan ini, satu objek Rekening dapat mengelola banyak objek Transaksi secara otomatis dan dinamis setiap kali terjadi aktivitas penyetoran maupun penarikan. Praktikum ini dilaksanakan untuk memberikan pemahaman praktis dalam menerjemahkan *Class Diagram* ke kode program Java serta manajemen koleksi objek di dalam objek lain.

1.2 Tujuan

1. Membaca dan menerjemahkan Class Diagram ke dalam kode program Java.
2. Memahami dan mengimplementasikan relasi Asosiasi (*one to many*)
3. Manajemen koleksi objek di dalam objek lain menggunakan tipe data bentukan dan struktur data dinamis (*ArrayList*).

1.3 Manfaat

1. Memudahkan pengubahan rancangan visual UML menjadi struktur kode Java secara akurat.
2. Memungkinkan satu objek mengelola banyak objek lain secara terstruktur seperti pada dunia nyata.

3. Memudahkan penyimpanan dan pengolahan data riwayat transaksi secara fleksibel tanpa batasan ukuran.

1.4 Link repository Github:

1. https://github.com/faridaarifin91-sudo/PrakPBO_B_2511531020/tree/master/PrakPBO/src/pekan2

BAB II

PEMBAHASAN

2.1 Dasar Teori

a. Class Diagram & Transformasi ke Kode Java

- *Class Diagram* merupakan bagian dari *Unified Modeling Language* (UML) yang menggambarkan struktur sistem dengan memperlihatkan class, atribut, method, serta hubungan antarclass. Penerjemahan diagram ini ke Java dilakukan dengan mendeklarasikan class, field/atribut, *constructor*, serta method sesuai dengan spesifikasi rancangan.

b. Relasi Asosiasi (*Has-A / One-to-Many*)

- Asosiasi adalah hubungan struktural yang menandakan bahwa suatu objek terhubung dengan objek lain.
- Relasi *One-to-Many* (1-ke-banyak) menunjukkan bahwa satu instansiasi dari sebuah class dapat berhubungan dengan banyak instansiasi dari class lainnya. Pada modul ini, hubungan *one-to-many* terjadi antara Rekening dan Transaksi (satu Rekening memiliki banyak Transaksi).

c. Struktur Data Dinamis (ArrayList)

- ArrayList adalah bagian dari framework *Java Collections* yang menyediakan array berukuran dinamis (*dynamic array*). Tidak seperti array konvensional berukuran tetap, ArrayList dapat bertambah ukurannya secara otomatis ketika data ditambahkan.
- Dalam relasi asosiasi, ArrayList digunakan untuk manajemen kumpulan objek kustom, misalnya ArrayList<Transaksi>, di mana instansiasi ArrayList wajib dilakukan pada *constructor* agar terhindar dari *error* NullPointerException.

d. Identifikasi Unik dan Penanda Waktu (*Timestamp*)

- Untuk membedakan setiap transaksi, dibuat atribut ID transaksi unik menggunakan kombinasi kode prefix (seperti "TRX-S-" untuk setor atau "TRX-T-" untuk tarik) dan `System.currentTimeMillis()` yang mengambil nilai milidetik dari waktu sistem berjalan.

2.2 Langkah kerja

2.2.1 Class 1 “Transaksi”

1. Kode Program

```
PrakPBO_B_2511531020 > src > pekan2 > Transaksi > Transaksi(String, String, double)
1 package pekan2;
2
3 public class Transaksi {
4     String idTransaksi;
5     String jenis;
6     double nominal;
7
8     // Constructor
9     public Transaksi(String id, String jenis, double nominal) {
10         this.idTransaksi = id;
11         this.jenis = jenis;
12         this.nominal = nominal;
13     }
14
15     public void cetakDetail() {
16         System.out.println("ID: " + idTransaksi + " | Jenis: " + jenis + " | Nominal: Rp" + nominal);
17     }
18 }
```

2. Analisis Langkah

Potongan Kode Program	Penjelasan Kode
<code>public class Transaksi {</code>	Membuat class bernama Transaksi yang berfungsi sebagai blueprint untuk menyimpan data transaksi.
<code>String idTransaksi;</code> <code>String jenis;</code>	Deklarasi 3 atribut utama: • <code>idTransaksi</code>: menyimpan ID unik transaksi. • <code>jenis</code>: menyimpan jenis transaksi ("Kredit" atau "Debit").

Potongan Kode Program	Penjelasan Kode
double nominal;	• nominal: menyimpan jumlah uang dengan tipe double agar dapat menampung nilai desimal.
<pre>public Transaksi(String id, String jenis, double nominal) { this.idTransaksi = id; this.jenis = jenis; this.nominal = nominal; }</pre>	Constructor class Transaksi yang dijalankan saat membuat objek baru. Kata kunci this digunakan untuk memasukkan nilai dari parameter ke atribut kelas.
<pre>public void cetakDetail() { System.out.println("ID: " + idTransaksi + " Jenis: " + jenis + " Nominal: Rp" + nominal); }</pre>	Method cetakDetail() yang bertugas menampilkan informasi lengkap satu baris transaksi ke konsol.

2.2.2 Class 2 “Rekening”

1. Kode Program

```
PrakPBO_B_2511531020 ▶ src ▶ pekan2 ▶ Rekening ▶ cetakMutasi(): void
1 package pekan2;
2
3 import java.util.ArrayList;
4
5 public class Rekening {
6     String nomorRekening;
7     String namaPemilik;
8     double saldo;
9
10    // Implementasi Asosiasi (1-to-many)
11    ArrayList<Transaksi> riwayatTransaksi;
12
13    // Constructor (Menyesuaikan Modul 1 & Modul 2)
14    public Rekening(String nomor, String nama, double saldoAwal) {
15        this.nomorRekening = nomor;
16        this.namaPemilik = nama;
17        this.saldo = saldoAwal;
18        // Wajib menginisialisasi ArrayList di dalam constructor agar tidak NullPointerException
19        this.riwayatTransaksi = new ArrayList<>();
20        System.out.println("Rekening atas nama " + namaPemilik + " berhasil dibuat dengan saldo Rp" + saldo);
21    }
22
23    // Method Getter nomorRekening (dari Modul 2)
24    public String getNomorRekening() {
25        return nomorRekening;
26    }
27
28    // Method Setor Tunai (Perekaman Otomatis Transaksi Kredit)
29    public void setorTunai(double nominal) {
30        if (nominal > 0) {
31            saldo += nominal;
32
33            // Merekam riwayat transaksi
34            String idTrx = "TRX-S-" + System.currentTimeMillis();
35
36            Transaksi trxBaru = new Transaksi(idTrx, "Kredit", nominal);
37            riwayatTransaksi.add(trxBaru);
38
39            System.out.println("Setor tunai Rp" + nominal + " berhasil. Saldo saat ini: Rp" + saldo);
40        } else {
41            System.out.println("Gagal: Nominal setor harus lebih dari 0!");
42        }
43    }
44
45    // TUGAS 1: Integrasi Rekam Jejak Penarikan (Tarik Tunai - Debit)
46    public void tarikTunai(double nominal) {
47        if (nominal >= 10000) {
48            if (saldo >= nominal) {
49                saldo -= nominal;
50
51                // Merekam riwayat transaksi penarikan
52                String idTrx = "TRX-T-" + System.currentTimeMillis();
53                Transaksi trxBaru = new Transaksi(idTrx, "Debit", nominal);
54                riwayatTransaksi.add(trxBaru);
55
56                System.out.println("Tarik tunai Rp" + nominal + " berhasil. Saldo saat ini: Rp" + saldo);
57            } else {
58                System.out.println("Transaksi Gagal: Saldo tidak mencukupi. Saldo Anda: Rp" + saldo);
59            }
60        } else {
61            System.out.println("Transaksi Gagal: Minimal nominal penarikan 10.000");
62        }
63    }
64
65    // Method Cek Informasi Rekening (Sesuai Modul 1)
66    public void cekInformasi() {
67        System.out.println("===== INFO REKENING =====");
68        System.out.println("No. Rekening : " + nomorRekening);
69        System.out.println("Nama Pemilik : " + namaPemilik);
70
71        System.out.println("Saldo Akhir : Rp" + saldo);
72        System.out.println("=====");
73    }
74
75    // TUGAS 2: Fitur Cetak Mutasi Rekening
76    public void cetakMutasi() {
77        System.out.println("\n===== MUTASI REKENING =====");
78        System.out.println("No. Rekening : " + nomorRekening);
79        System.out.println("Nama Pemilik : " + namaPemilik);
80        System.out.println("-----");
81
82        if (riwayatTransaksi.isEmpty()) {
83            System.out.println("Belum ada transaksi pada rekening ini.");
84        } else {
85            // Perulangan for-each untuk menelusuri riwayat transaksi
86            for (Transaksi t : riwayatTransaksi) {
87                t.cetakDetail();
88            }
89            System.out.println("-----");
90        }
91    }
92 }
```

2. Analisis Langkah

Potongan Kode Program	Penjelasan Kode
import java.util.ArrayList;	Mengimpor class ArrayList dari paket java.util agar dapat menggunakan struktur data dinamis untuk menyimpan daftar transaksi.
public class Rekening { String nomorRekening; String namaPemilik; double saldo;	Membuat class Rekening beserta atribut dasar penyimpan nomor rekening, nama pemilik, dan saldo.
ArrayList<Transaksi> riwayatTransaksi;	Mengimplementasikan relasi Asosiasi (<i>one-to-many</i>). Atribut ini berupa ArrayList yang digunakan sebagai tempat menampung kumpulan objek Transaksi.
public Rekening(String nomor, String nama, double saldoAwal) { this.nomorRekening = nomor; this.namaPemilik = nama; this.saldo = saldoAwal;	Constructor class Rekening. Baris this.riwayatTransaksi = new ArrayList<>() sangat penting dieksekusi di dalam constructor untuk menginisialisasi wadah penampung agar tidak terjadi NullPointerException saat transaksi ditambahkan.

Potongan Kode Program	Penjelasan Kode
<pre> this.riwayatTransaksi = new ArrayList<>(); System.out.println("Reke ning atas nama " + namaPemilik + " berhasil dibuat dengan saldo Rp" + saldo); } </pre>	
<pre> public String getNomorRekening() { return nomorRekening; } </pre>	Method <i>getter</i> untuk mengambil dan mengembalikan nilai dari variabel nomorRekening.
<pre> Public void setorTunai(double nominal) { if (nominal > 0) { saldo += nominal; } } </pre>	Method untuk melakukan penyetoran: 1. Memeriksa apakah nominal lebih dari 0. 2. Mengupdate nilai saldo (saldo += nominal).

Potongan Kode Program	Penjelasan Kode
<pre>String idTrx = "TRX-S-" + System.currentTimeMillis ()); Transaksi trxBaru = new Transaksi(idTrx, "Kredit", nominal); riwayatTransaksi.add(trx Baru); System.out.println("Setor tunai Rp" + nominal + " berhasil. Saldo saat ini: Rp" + saldo); } else { System.out.println("Gagal : Nominal setor harus lebih dari 0!"); } }</pre>	3. Membangkitkan ID unik transaksi berdasarkan timestamp. 4. Membuat objek Transaksi baru bertipe "Kredit" lalu menyimpannya ke riwayatTransaksi dengan method .add().
<pre>public void tarikTunai(double nominal) { if (nominal < 10000) {</pre>	Method penarikan uang yang memenuhi validasi: 1. Mengecek batas minimal penarikan (Rp10.000).

Potongan Kode Program	Penjelasan Kode
<pre> System.out.println("Trans aksi Gagal: Minimal penarikan Rp10.000"); } else if (nominal > saldo) { System.out.println("Trans aksi Gagal: Saldo tidak mencukupi. Saldo Anda: Rp" + saldo); } else { saldo -= nominal; String idTrx = "TRX-T-" + System.currentTimeMillis (); Transaksi trxBaru = new Transaksi(idTrx, "Debit", nominal); riwayatTransaksi.add(trx Baru); </pre>	2. Mengecek apakah saldo mencukupi penarikan. 3. Jika lolos validasi, saldo dikurangi (saldo -= nominal). 4. Objek Transaksi bertipe "Debit" diinstansiasi dan dimasukkan ke riwayatTransaksi.

Potongan Kode Program	Penjelasan Kode
<pre> System.out.println("Tarik tunai Rp" + nominal + " berhasil. Saldo saat ini: Rp" + saldo); } } </pre>	
<pre> public void cekInformasi() { System.out.println("==== ===== INFO REKENING ====="); System.out.println("No. Rekening : " + nomorRekening); System.out.println("Nama Pemilik : " + namaPemilik); System.out.println("Saldo Akhir : Rp" + saldo); System.out.println("==== ===== ===== ====="); </pre>	Method untuk menampilkan ringkasan data pemilik dan saldo akhir dari objek rekening yang dipanggil.

Potongan Kode Program	Penjelasan Kode
<pre> }</pre>	
<pre> public void cetakMutasi() { System.out.println("\n== ===== MUTASI REKENING ====="); System.out.println("No. Rekening : " + nomorRekening); System.out.println("Nama Pemilik : " + namaPemilik); System.out.println("----- ----- -----"); if (riwayatTransaksi.isEmpty y()) { System.out.println("Belu m ada transaksi pada rekening ini."); } }</pre>	Method untuk mencetak mutasi: 1. Mengecek riwayatTransaksi.isEmpty(). Jika true, menampilkan pesan belum ada transaksi. 2. Jika terdapat data, dilakukan perulangan <i>for-each</i> untuk memanggil t.cetakDetail() pada setiap objek transaksi di dalam list.

Potongan Kode Program	Penjelasan Kode
<pre> } else { for (Transaksi t : riwayatTransaksi) { t.cetakDetail(); } } System.out.println("----- ----- -----"); } </pre>	

2.2.3 Class 3 “Main”

1. Kode Program

```

PrakPBO_B_2511531020 ▶ src ▶ pekan2 ▶ Main ▶ main(String[]) : void
1 package pekan2;
2
3 import java.util.ArrayList;
4
5
6 public class Main {
7     public static void main(String[] args) {
8         Scanner input = new Scanner(System.in);
9         Rekening akunAktif = null; // Objek belum diinisialisasi (Sesuai Modul 1)
10        boolean isRunning = true;
11
12        System.out.println("=== SISTEM PERBANKAN MINI ===");
13
14        while (isRunning) {
15            System.out.println("\nMenu Utama:");
16            System.out.println("1. Buka Rekening Baru");
17            System.out.println("2. Setor Tunai");
18            System.out.println("3. Tarik Tunai");
19            System.out.println("4. Cek Informasi Rekening");
20            System.out.println("6. Cetak Mutasi (Riwayat)");
21            System.out.println("0. Keluar");
22            System.out.print("Pilih menu: ");
23
24            int pilihan = input.nextInt();
25            input.nextLine(); // Clears buffer
26
27            switch (pilihan) {
28                case 1:
29                    System.out.print("Masukkan No Rekening: ");
30                    String no = input.nextLine();
31                    System.out.print("Masukkan Nama Pemilik: ");
32                    String nama = input.nextLine();
33                    System.out.print("Masukkan Saldo Awal: ");
34                    double saldo = input.nextDouble();
35

```

```

36 // Instansiasi Object / Penjelasan Constructor
37 akunAktif = new Rekening(no, nama, saldo);
38 break;
39
40 case 2:
41     if (akunAktif == null) {
42         System.out.println("Error: Mohon maaf, Anda belum memiliki nomor rekening!");
43     } else {
44         System.out.print("Masukkan nominal setor: ");
45         double setor = input.nextDouble();
46         akunAktif.setorTunai(setor);
47     }
48     break;
49
50 case 3:
51     if (akunAktif == null) {
52         System.out.println("Error: Mohon maaf, Anda belum memiliki nomor rekening!");
53     } else {
54         System.out.print("Masukkan nominal tarik: ");
55         double tarik = input.nextDouble();
56         akunAktif.tarikTunai(tarik);
57     }
58     break;
59
60 case 4:
61     if (akunAktif == null) {
62         System.out.println("Error: Anda belum membuka rekening!");
63     } else {
64         akunAktif.cekInformasi();
65     }
66     break;
67
68 case 6:
69     // TUGAS 2: Panggilan method cetakMutasi()
70     // Panggilan method cetakMutasi()
71     if (akunAktif == null) {
72         System.out.println("Error: Anda belum membuka rekening!");
73     } else {
74         akunAktif.cetakMutasi();
75     }
76     break;
77
78 case 0:
79     isRunning = false;
80     System.out.println("Sistem ditutup. Terima kasih!");
81     break;
82
83 default:
84     System.out.println("Pilihan tidak valid!");
85 }
86 input.close();
87 }
88 }

```

2. Analisis Langkah

Potongan Kode Program	Penjelasan Kode
<pre>import java.util.Scanner; public class Main { public static void main(String[] args) {</pre>	Mengimpor Scanner untuk input pengguna, membuat kelas Main, dan mendefinisikan main() sebagai method utama yang dieksekusi pertama kali.
<pre>Scanner input = new Scanner(System.in);</pre>	Inisialisasi komponen utama:

Potongan Kode Program	Penjelasan Kode
<pre>Rekening akunAktif = null; boolean isRunning = true;</pre>	- Objek Scanner bernama input. - Variabel <code>akunAktif</code> bertipe <code>Rekening</code> dengan nilai awal <code>null</code> (belum ada rekening aktif). - Variabel <code>isRunning = true</code> sebagai pengontrol perulangan menu.
<pre>while (isRunning) { System.out.println("\nMenu Utama:"); System.out.println("1. Buka Rekening Baru"); System.out.println("2. Setor Tunai"); System.out.println("3. Tarik Tunai");</pre>	Menjalankan perulangan <code>while</code> selama <code>isRunning</code> bernilai <code>true</code> dan menampilkan pilihan menu utama ke konsol.

Potongan Kode Program	Penjelasan Kode
<pre>System.out.println("4. Cek Informasi Rekening"); System.out.println("6. Cetak Mutasi (Riwayat)"); System.out.println("0. Keluar"); System.out.print("Pilih menu: ");</pre>	
<pre>int pilihan = input.nextInt(); input.nextLine();</pre>	Membaca input nomor menu yang dipilih pengguna dan mengeksekusi <code>input.nextLine()</code> untuk membersihkan karakter <i>newline</i> (Enter) dari buffer.
<pre>switch (pilihan) {</pre>	Struktur percabangan <code>switch</code> untuk mengeksekusi blok kode sesuai dengan nilai variabel <code>pilihan</code>.
Case 1: Buka Rekening <pre>case 1: System.out.print("Masukkan No Rekening: "); String no = input.nextLine();</pre>	Membaca input nomor rekening, nama pemilik, dan saldo awal. Kemudian menginstansiasi objek Rekening baru dan menyimpannya ke variabel <code>akunAktif</code>.

Potongan Kode Program	Penjelasan Kode
<pre> System.out.print("Masukkan Nama Pemilik: "); String nama = input.nextLine(); System.out.print("Masukkan Saldo Awal: "); double saldo = input.nextDouble(); akunAktif = new Rekening(no, nama, saldo); break; </pre>	
Case 2: Setor Tunai <pre> case 2: if (akunAktif == null) { System.out.println("Error: Mohon maaf, Anda belum memiliki nomor rekening!"); } else { </pre>	Memeriksa ketersediaan akunAktif. Jika null, menampilkan pesan error. Jika tidak null, membaca input nominal lalu memanggil method akunAktif.setorTunai(setor).

Potongan Kode Program	Penjelasan Kode
<pre> System.out.print("Masukkan nominal setor: "); double setor = input.nextDouble(); akunAktif.setorTunai(setor); } break; </pre>	
Case 3: Tarik Tunai <pre> case 3: if (akunAktif == null) { System.out.println("Error: Mohon maaf, Anda belum memiliki nomor rekening!"); } else { System.out.print("Masukkan nominal tarik: "); double tarik = input.nextDouble(); akunAktif.tarikTunai(tarik); } </pre>	Memeriksa ketersediaan rekening. Jika rekening ada, membaca nominal tarik tunai lalu memanggil method <code>akunAktif.tarikTunai(tarik)</code>.

Potongan Kode Program	Penjelasan Kode
break;	
Case 4: Cek Informasi case 4: <pre> if (akunAktif == null) { System.out.println("Error: Anda belum membuka rekening!"); } else { akunAktif.cekInformasi(); } break; </pre>	Mengecek apakah rekening sudah dibuat. Jika ada, mengeksekusi <code>akunAktif.cekInformasi()</code> untuk menampilkan detail saldo dan pemilik.
Case 6: Cetak Mutasi case 6: <pre> if (akunAktif == null) { System.out.println("Error: Anda belum membuka rekening!"); } else { akunAktif.cetakMutasi(); </pre>	Memeriksa ketersediaan rekening. Jika ada, mengeksekusi <code>akunAktif.cetakMutasi()</code> untuk mencetak seluruh riwayat transaksi.

Potongan Kode Program	Penjelasan Kode
<pre> } break; </pre>	
Case 0: Keluar <pre> case 0: isRunning = false; System.out.println("Sistem ditutup. Terima kasih!"); break; </pre>	Mengubah nilai isRunning menjadi false sehingga perulangan while terhenti dan menampilkan pesan penutupan sistem.
<pre> default: System.out.println("Pilihan tidak valid!"); } </pre>	Menangani kondisi jika pengguna memasukkan nomor menu selain pilihan yang tersedia.
<pre> input.close(); </pre>	Menutup objek Scanner untuk membebaskan sumber daya memori setelah program selesai dijalankan.

BAB III

KESIMPULAN

Praktikum ini berhasil mengimplementasikan konsep dasar Pemrograman Berbasis Objek (PBO), seperti pembuatan *class*, pengisian nilai melalui *constructor*, dan penggunaan method *getter* untuk pengaksesan data. Penerapan relasi Asosiasi (*one-to-many*) antara kelas Rekening dan Transaksi menggunakan struktur data dinamis ArrayList terbukti efektif, sehingga satu rekening dapat menyimpan dan mengelola banyak data riwayat transaksi secara terstruktur.

Selain itu, sistem perbankan berbasis CLI yang dibangun mampu menjalankan logika bisnis dengan aman melalui berbagai mekanisme validasi. Program berhasil membatasi penarikan di bawah nominal minimal (Rp10.000), mencegah transaksi melampaui saldo aktif, serta menolak perintah jika rekening belum terdaftar. Penggunaan struktur kontrol while dan switch-case pada kelas utama juga menghasilkan antarmuka berbasis menu yang interaktif dan responsif terhadap input pengguna.

DAFTAR PUSTAKA

Nurfiah. (2026). *Modul Praktikum Pemrograman Berorientasi Objek: Relasi Antar Objek (Class Diagram & Asosiasi)*. Laboratorium Komputer, Departemen Informatika, Universitas Andalas.